

Ontwikkeling van een smart- phone sdk voor het ob- fusceren van locatie data

Milan SCHOLLIER

Promotor(en): prof. dr. Vincent Naessen Masterproef ingediend tot het behalen van
de graad van master of Science in de
Co-promotor(en): dr. ing. Michiel Willocx industriële wetenschappen: Master in de
industriële wetenschappen elektronica-ICT:
smart applications

Academiejaar 2024 - 2025

©Copyright KU Leuven

Deze masterproef is een examendocument dat niet werd gecorrigeerd voor eventuele vastgestelde fouten.

Zonder voorafgaande schriftelijke toestemming van zowel de promotor(en) als de auteur(s) is overnemen, kopiëren, gebruiken of realiseren van deze uitgave of gedeelten ervan verboden. Voor aanvragen i.v.m. het overnemen en/of gebruik en/of realisatie van gedeelten uit deze publicatie, kan u zich richten tot KU Leuven Technologicampus Gent, Gebroeders De Smetstraat 1, B-9000 Gent, +32 92 65 86 10 of via e-mail iiw.gent@kuleuven.be.

Voorafgaande schriftelijke toestemming van de promotor(en) is eveneens vereist voor het aanwenden van de in deze masterproef beschreven (originele) methoden, producten, schakelingen en programma's voor industrieel of commercieel nut en voor de inzending van deze publicatie ter deelname aan wetenschappelijke prijzen of wedstrijden.

Dankwoord

Voor u ligt de thesis "Ontwikkeling van een smartphone sdk voor het obfusceren van locatie data". Deze thesis is geschreven om te voldoen aan de afstudeereisen van de Master in de industriële wetenschappen elektronica-ICT: smart applications aan de Katholieke Universiteit van Leuven.

Ervaringen:

Dankwoord:

Ik hoop dat het onderwerp u even hard kan boeien als mij.

Milan Schollier

Gent, May 16, 2025



*Copyright informatie:
studiewerk van een student in het kader van de
academische opleiding en examenbeoordeling.
Na deze beoordeling vond geen correctie plaats
van het studiewerk.*

Abstract

In today's data-driven economy, location data holds immense value. It is used to gain meaningful crowd insights and optimize various sectors of society, ranging from tourism to retail. However, the methods used to collect these valuable location traces often occur without the user's explicit, informed consent. The high precision of location data and the frequency of reports pose significant privacy risks for individuals.

These risks include not only the inference of sensitive locations, such as home, workplace, and leisure activities, but also the construction of social graphs. In the hands of malicious actors, this information could lead to blackmail, coercion, or hate crimes.

To address these privacy concerns, this thesis proposes a privacy-friendly alternative for collecting valuable location traces. A novel Android library is developed that obfuscates both locations and timestamps by introducing noise, ensuring that individual traces can no longer be linked to a specific user. This prevents sensitive information from leaking while preserving the utility of the data. Importantly, the library operates entirely on-device, ensuring that the real, accurate traces never leave the device.

This achieve this, this thesis explores the Android ecosystem on lifecycles and background tasks to design and implement advanced location obfuscation algorithms within the library.

A set of experiments is conducted to demonstrate the effectiveness of the solution. Among other evaluations, this thesis compares the proposed library to the Approximate Location feature introduced in Android 12, offering insights into advancements in mobile privacy measures.

In de huidige data-gedreven economie heeft locatiedata een enorme waarde. Het wordt gebruikt om waardevolle inzichten te verkrijgen over menigten en om verschillende sectoren van de samenleving te optimaliseren, variërend van toerisme tot retail.

De methoden die worden gebruikt om deze waardevolle locatiegegevens te verzamelen, gebeuren echter vaak zonder de expliciete, geïnformeerde toestemming van de gebruiker. De hoge precisie van locatiedata en de frequentie van rapportages brengen aanzienlijke privacyrisico's met zich mee voor individuen.

Deze risico's omvatten niet alleen het afleiden van gevoelige locaties, zoals thuis, werkplek en vrijetijdsactiviteiten, maar ook de constructie van sociale grafen. In handen van kwaadwillende actoren kan deze informatie leiden tot chantage, dwang of haatmisdrijven.

Om deze privacyproblemen aan te pakken, stelt deze thesis een privacyvriendelijk alter-

natief voor het verzamelen van waardevolle locatiegegevens voor. Een innovatieve Android-bibliotheek is ontwikkeld die zowel locaties als tijdstempels obfusceert door ruis toe te voegen, waardoor individuele sporen niet langer aan een specifieke gebruiker kunnen worden gekoppeld. Dit voorkomt dat gevoelige informatie uitlekt, terwijl de bruikbaarheid van de data behouden blijft. Belangrijk is dat de bibliotheek volledig op het apparaat werkt, waardoor de echte, nauwkeurige gegevens het apparaat nooit verlaten.

Om dit te bereiken, onderzoekt deze thesis het Android-ecosysteem, inclusief levenscycli en achtergrondtaken, om geavanceerde algoritmen voor locatie-obfuscatie te ontwerpen en te implementeren binnen de bibliotheek.

Een reeks experimenten is uitgevoerd om de effectiviteit van de oplossing aan te tonen. Onder andere evaluaties vergelijkt deze thesis de voorgestelde bibliotheek met de Approximate Location-functie die is geïntroduceerd in Android 12, en biedt inzichten in de vooruitgang op het gebied van mobiele privacymaatregelen.

Trefwoorden: Android library, location obfuscation, location tracking

Contents

1	Inleiding	3
1.1	probleemstelling	3
1.2	onderzoeksvragen	4
1.3	Doelen en scope	4
1.4	Overzicht van de thesis	4
2	State of the art	7
2.1	Android besturingssysteem	7
2.1.1	library	7
2.1.2	activity	7
2.1.3	services	9
2.2	Locatie obfuscatie	10
2.2.1	Android's oplossing	10
2.2.2	Voorgaand onderzoek	11
2.3	Preliminair experimenten	11
2.3.1	Verschillende nauwkeurigheidsniveaus	12
2.3.2	Coarse location Android	12
3	Design	17
3.1	Vereisten	17
3.2	Veronderstellingen	18
3.3	Ontwerp	18
4	Implementatie	21
4.1	Overzicht library	21
4.1.1	Locatie tracking	21
4.1.2	Obfuscatie	24
4.1.3	Opslag	25

4.1.4	Netwerk	25
4.2	Dataverwerking	26
4.2.1	Verzameling	26
4.2.2	verwerking	26
4.3	Conclusie	26
5	Resultaten	29
5.1	Visuele representatie	29
6	Evaluatie en discussie	31
6.1	Evaluatie van het systeem	31
6.1.1	beperkingen	31
6.2	privacy	32
6.3	Gebruik van de library	32
6.3.1	initialisatie	32
6.3.2	LocationObfuscator zelf implementeren	32
6.3.3	Live data	33
6.4	Conclusie	33
7	Conclusie	35
A	Code van het obfuscatie algoritme	37
B	Beschrijving van deze masterproef in de vorm van een wetenschappelijk artikel	39
C	Poster	41

List of Figures

2.1	Activity lifecycle Android	8
2.2	Android WorkManager	10
2.3	Vergelijking coarse permissies en fine permissies	14
2.4	Vergelijking coarse permissie op één apparaat	14
2.5	Vergelijking coarse permissie op meerdere apparaten	15
3.1	Flow diagram van continue locatieverzameling	19
4.1	Klasse diagram van de library	22
4.2	Flow diagram van het obfuscatie algoritme	25
5.1	Obfuscatiealgoritme gevisualiseerd	29

List of Tables

Acronyms

AAR Android archive. 7

APK Android package. 7

1

Inleiding

Veel moderne applicaties verzamelen de locatiegegevens van hun gebruikers, deze locatiegegevens kunnen dan gebruikt worden voor crowd-insights dat nuttig is voor verscheidene onderzoeken, zoals mobiliteitsstudies en stadsplanning. Echter, de hoge nauwkeurigheid en frequentie van deze gegevens brengen aanzienlijke privacyrisico's met zich mee. In verkeerde handen kunnen locatiegegevens leiden tot sociale profilering of zelfs spionage, wat de persoonlijke veiligheid van gebruikers in gevaar kan brengen.

Met deze thesis wordt een bijdrage geleverd aan de ontwikkeling van privacyvriendelijke technologieën voor mobiele toepassingen. De voorgestelde oplossing verlaagt de instapdrempel voor ontwikkelaars en bevordert het veilig gebruik van locatiegegevens in een wereld waarin data steeds waardevoller — en kwetsbaarder — wordt.

1.1 probleemstelling

Het centrale probleem is dat de werkelijke locatie van een gebruiker doorgaans rechtstreeks naar de server wordt verzonden en daar zonder enige vorm van obfuscatie wordt opgeslagen. Hierdoor blijven locatiegegevens in hun oorspronkelijke, niet-versluierde vorm beschikbaar, wat aanzienlijke privacyrisico's met zich meebrengt. Bovendien worden deze gegevens vaak zonder verdere bewerking doorverkocht aan data brokers.

Om de privacy van gebruikers te waarborgen, moeten verschillende voorwaarden worden vervuld. Ten eerste moet de overdracht van gegevens naar de server beveiligd zijn tegen onderschepping. Ten tweede dient de server zelf adequaat beschermd te zijn tegen datalekken. Daarnaast is het essentieel dat opgeslagen locatiegegevens op een geobfusceerde manier

worden bewaard en dat eventuele doorverkoop van deze gegevens eveneens uitsluitend in geobfusceerde vorm plaatsvindt. Tot slot is een strikte controle op de partijen die toegang krijgen tot deze gegevens noodzakelijk om misbruik te voorkomen.

De aanwezigheid van deze potentiële kwetsbaarheden bemoeilijkt het voor gebruikers om vertrouwen te hebben in de huidige verwerking van locatiegegevens.

Bovendien vormt de steile leercurve van de relevante Android-technologieën een bijkomende uitdaging. Dit kan, zeker bij beperkte ervaring of strakke deadlines, leiden tot fouten of beveiligingslacunes in de implementatie. Een goed ontworpen bibliotheek die de complexiteit voor ontwikkelaars verlaagt en standaard voorziet in robuuste beveiligingsmaatregelen, kan deze risico's aanzienlijk beperken.

Dus deze problemen maken een mooie grondlegging als thesisonderwerp.

1.2 onderzoeksvragen

Het onderzoek beantwoordt drie kernvragen: (1) Hoe kan een Android-bibliotheek worden ontworpen die locatiegegevens lokaal en gebruiksvriendelijk obfusceert? (2) In hoeverre beschermd de obfuscatie de locatie van de gebruiker?

1.3 Doelen en scope

Het doel van deze thesis is om een data stream obfuscator te implementeren als een Android-bibliotheek. Deze obfuscator voegt ruis toe aan zowel de locatie- als tijdgegevens, zodat individuele bewegingspatronen niet meer naar specifieke personen kunnen worden herleid. De ontwikkeling van deze bibliotheek zou de obfuscatie berekening van de server naar de client verhuizen. Waardoor gevoelige informatie (bv. een privacy bubbel) lokaal op het apparaat blijven en enkel geobfusceerde locaties worden doorgestuurd en zo data lekken minder schadelijk zijn.

In deze thesis wordt de focus uitsluitend gelegd op het lokaal verzamelen en obfusceren van locatie- en tijdgegevens op Android-apparaten. Aspecten zoals server-side beveiliging, netwerkbeveiliging en bredere privacyvraagstukken, waaronder identiteitsbeheer en gebruiker-sconsent, worden buiten beschouwing gelaten. De implementatie is beperkt tot het Android-platform; andere mobiele besturingssystemen, zoals iOS, vallen buiten de scope van dit onderzoek.

1.4 Overzicht van de thesis

Het verloop van thesis is als volgt:

Hoofdstuk 2 State of the art: In dit hoofdstuk wordt een overzicht gegeven van de relevante aspecten van het Android besturingssysteem die van belang zijn voor deze thesis. Daarnaast wordt de huidige stand van zaken omtrent locatieverzameling en obfuscatie besproken. Ook worden enkele preliminaire experimenten toegelicht die bijdragen aan een beter begrip van de implementatie in Android, wat de basis vormt voor de keuzes die in Hoofdstuk 4 worden gemaakt.

Hoofdstuk 3 Design: In dit hoofdstuk worden de vereisten en veronderstellingen van de bibliotheek besproken. Daarnaast worden de belangrijkste ontwerpkeuzes en architecturale beslissingen toegelicht die aan de basis liggen van de implementatie.

Hoofdsuk 4 Implementatie: In dit hoofdstuk wordt de volledige werkwijze van implementatie toegelicht.

Hoofdsuk 5 Resultaten: In dit hoofdstuk wordt visueel en numeriek de resultaten van de gemaakte library toegelicht.

Hoofdsuk 6 Evaluatie en discussie: Evaluatie of de bibliotheek aan de vereisten voldoet, mogelijke uitbreidingen en mogelijke verbeteringen

2

State of the art

Voor de achtergrond te kaderen voor het werk van deze thesis te kunnen bespreken zijn er 2 grote stromingen: Android besturingssysteem en locatieverzameling/-obfuscatie. Ook is er een extra sectie voorzien voor het onderzoek naar hoe de standaardimplementatie van obfuscatie werkt in Android.

2.1 Android besturingssysteem

2.1.1 library

Voor de efficiënte implementatie van code door ontwikkelaars, zonder herhaaldelijk gebruik te maken van copy-paste, worden libraries gebruikt. Android-applicaties worden geprogrammeerd in Java of Kotlin, waarbij de build-tool Gradle wordt ingezet. Deze build-tool beheert de installatie van de benodigde libraries, zodat deze eenvoudig beschikbaar zijn voor integratie in de eigen code. In tegenstelling tot een standaardapplicatie, die wordt gecompileerd naar een Android package (APK), wordt een library gecompileerd naar een Android archive (AAR). Deze AAR-bestanden kunnen vervolgens worden gepubliceerd naar publieke Gradle-plugins, waardoor ze eenvoudig herbruikbaar zijn in andere projecten.

2.1.2 activity

Een activity in Android is een enkele, gefocuste taak die een gebruiker kan uitvoeren. Het vormt een van de fundamentele bouwstenen van een Android-applicatie. Elke activity heeft

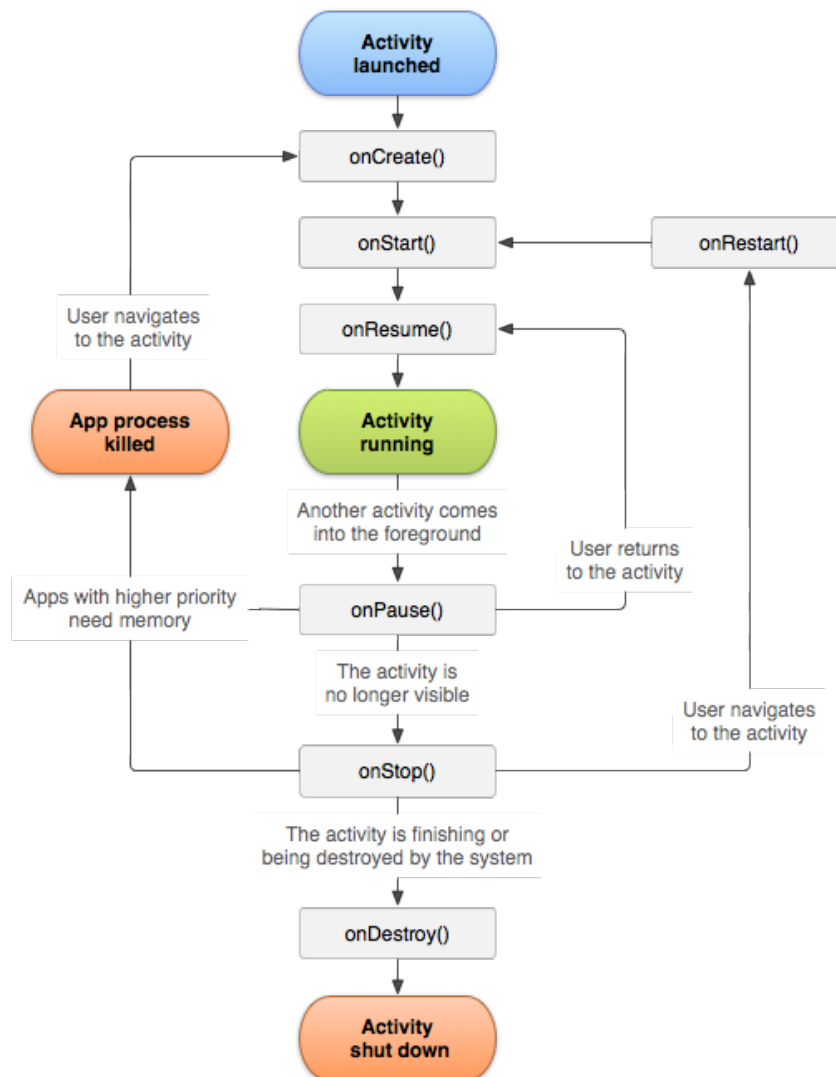


Figure 2.1: Activity lifecycle Android

een eigen lifecycle, die wordt beheerd door het Android-systeem. Deze lifecycle bestaat uit verschillende states, zoals onCreate, onStart, onResume, onPause, onStop en onDestroy.

Bij het ontwikkelen van een applicatie is het belangrijk om rekening te houden met deze lifecycle-methoden, omdat ze bepalen hoe de applicatie reageert op gebruikersinteracties en systeemgebeurtenissen. Bijvoorbeeld, in de onCreate-methode wordt de initiële setup van de activity uitgevoerd, zoals het instellen van de gebruikersinterface. De onPause- en onStop-methoden worden aangeroepen wanneer de activity naar de achtergrond gaat, en resources kunnen hier worden vrijgegeven om geheugen te besparen.

De mogelijkheden binnen de thread van een activity zijn beperkt. Langdurige taken worden bijvoorbeeld abrupt beëindigd wanneer de activity naar de achtergrond verdwijnt. Om dit probleem te omzeilen, biedt Android alternatieve oplossingen, die verderop in deze thesis

worden besproken. Daarnaast is het niet toegestaan om blocking calls, zoals een HTTP-request of het opvragen van locatiegegevens via de Location Provider, uit te voeren in de main thread. Dit is noodzakelijk om te voorkomen dat de gebruikersinterface vastloopt en de gebruikservaring negatief wordt beïnvloed. Zo crashed de applicatie als de main thread wordt geblocked voor 5 seconden.

2.1.3 services

Om de beperkingen van een activity, die slechts kort op de achtergrond actief kan blijven, te omzeilen, biedt Android verschillende tools die door de jaren heen zijn geëvolueerd. Deze tools maken het mogelijk om taken op de achtergrond uit te voeren, zelfs wanneer de applicatie niet actief is. De belangrijkste tools zijn de AlarmManager, coroutines en de WorkManager. Hieronder worden deze tools besproken, samen met hun voor- en nadelen.

AlarmManager De AlarmManager is een mechanisme dat het apparaat uit de zogenaamde 'doze mode' of energiebesparende modus kan halen. Dit maakt het mogelijk om op specifieke tijdstippen een taak uit te voeren. Echter, de AlarmManager heeft enkele beperkingen. Zo kan het slechts één keer per uur een taak uitvoeren en is het niet geschikt voor langdurige taken. Dit maakt het minder geschikt voor toepassingen die continu of frequent achtergrondtaken moeten uitvoeren.

Coroutines Coroutines kunnen worden gezien als een lichtgewicht alternatief voor threads. Ze bieden een efficiënte manier om asynchrone taken uit te voeren zonder de main thread te blokkeren. Echter, coroutines zijn gekoppeld aan de lifecycle van de aanroepende activity. Dit betekent dat wanneer de activity wordt gestopt, ook de coroutines worden beëindigd. Hierdoor zijn coroutines niet geschikt voor persistent werk dat moet blijven draaien, ongeacht de status van de activity.

WorkManager De WorkManager is een krachtig en flexibel API voor het uitvoeren van achtergrondtaken. Het biedt ondersteuning voor zowel eenmalige (one-time) als periodieke (periodic) taken. Voor achtergrondtaken die regelmatig moeten worden uitgevoerd, zoals het verzamelen van locatiegegevens, is de periodieke versie geschikt. Hierbij kan een taak bijvoorbeeld elke 15 minuten worden uitgevoerd, met een maximale duur van 10 minuten per taak. Voor taken die slechts één keer hoeven te worden uitgevoerd, zoals een langdurige upload, kan de eenmalige versie worden gebruikt. Het is echter belangrijk op te merken dat langdurige taken in de foreground een permanente notificatie in het besturingssysteem vereisen, wat de gebruiker kan storen.

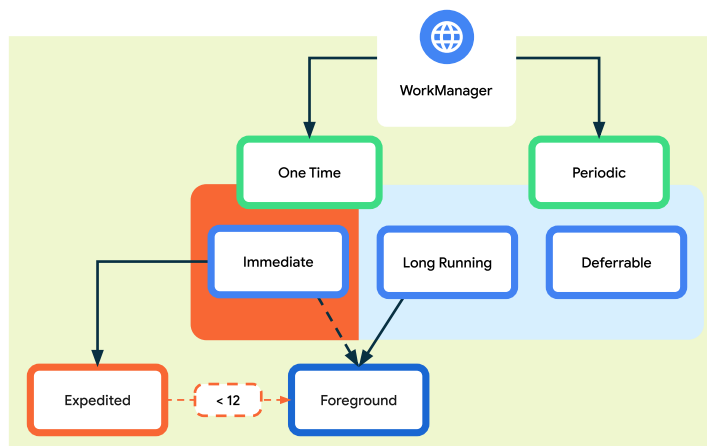


Figure 2.2: Android WorkManager

2.2 Locatie obfuscatie

In de studie van obfuscatie heb je 2 paden: hoe Android het aanpakt en voorgaand onderzoek losstaand van Android.

2.2.1 Android's oplossing

Voordat er dieper wordt ingegaan op locatie-obfuscatie in Android, is het belangrijk om de werking van de locatievoorzieningen in Android te bespreken. Android biedt twee primaire opties om locatiegegevens op te vragen: de standaard open-source implementatie en de Google Play Services-implementatie.

De standaard open-source implementatie leest enkel de data uit van de sensoren en voert geen verdere berekeningen of obfuscatie-algoritmen uit. Deze aanpak biedt ontwikkelaars meer vrijheid, maar brengt ook enkele nadelen met zich mee. Zo worden applicaties die deze versie gebruiken vaker gemarkeerd als potentieel kwaadaardig, omdat ze minder geïntegreerd zijn in het Android-ecosysteem.

De Google Play Services-implementatie, daarentegen, maakt gebruik van de Fused Location Provider. De Fused Location Provider combineert gegevens van verschillende sensoren, zoals GPS, wifi, mobiele netwerken en zelfs bewegingssensoren, om een zo nauwkeurig mogelijke locatie te berekenen. Dit proces wordt 'sensor fusion' genoemd. De Fused Location Provider biedt ontwikkelaars een eenvoudige API om locatiegegevens op te vragen, waarbij automatisch de meest geschikte bron wordt gekozen op basis van de context, zoals batterijverbruik, nauwkeurigheid en beschikbaarheid van sensoren. Bovendien ondersteunt de Fused Location Provider zowel real-time locatie-updates als eenmalige locatieverzoeken, wat het een flexibele oplossing maakt voor uiteenlopende toepassingen.

In de Fused Location Provider zijn er daarnaast verschillende methoden om locatiegegevens

te verkrijgen. De meest nauwkeurige methode is via GPS-satellieten, maar alternatieve methoden zoals het gebruik van lokale telefoonmasten of wifi-netwerken bieden ook mogelijkheden. Deze alternatieve methoden introduceren een inherente vorm van obfuscatie, aangezien de nauwkeurigheid doorgaans lager is dan die van GPS. Echter, in dichtbevolkte stedelijke gebieden kan dit juist leiden tot een hogere nauwkeurigheid dan GPS, omdat gebouwen GPS-signalen kunnen blokkeren.

Sinds Android 12 hebben gebruikers de mogelijkheid gekregen om alleen toegang tot een benaderde locatie (approximate location) toe te staan, wat een extra dimensie toevoegt aan de discussie over privacy en nauwkeurigheid. De coarse location-functionaliteit in Android maakt geen gebruik van GPS-satellieten en voegt bovendien een extra laag van obfuscatie toe aan de locatiegegevens. Hoe deze obfuscatie precies wordt toegepast, is echter niet publiekelijk gedocumenteerd.

2.2.2 Voorgaand onderzoek

In eerdere onderzoeken, uitgevoerd door Kevin De Boeck, werd aangetoond dat locatie-obfuscatie een dringende noodzaak is. Uit data-analyse op een dataset van locatiegegevens in Frankrijk, verzameld over een periode van zeven dagen, bleek dat gevoelige locaties eenvoudig konden worden geïdentificeerd. Daarnaast konden sociale netwerken worden gereconstrueerd en konden plaatsen van interesse voor individuen gemakkelijk in kaart worden gebracht.

Voortbouwend op deze bevindingen heeft Toon Dehaene onderzoek gedaan naar methoden om GPS-locaties te obfusceren. Het voorgestelde algoritme introduceerde het concept van privacycirkels op locaties die niet eerder waren bezocht. Wanneer een gebruiker zich opnieuw binnen dezelfde cirkel bevindt, wordt altijd het middelpunt van deze cirkel geretourneerd. Dit proces voegt ruis toe aan de coördinaten. Bovendien werd er variatie toegevoegd aan de tijdstippen van locatiegegevens. Dit algoritme werd geïmplementeerd in Rust en toegepast aan de serverzijde om locatie-obfuscatie te realiseren. Het algoritme vormde tevens de basis voor de standaardimplementatie van obfuscatie in de Android-library wat verder wordt uitgelegd.

In vervolgonderzoeken werd verder geëxperimenteerd met het gridsnapping van locaties aan lokale wegen. Dit had als doel om de obfuscatie realistischer te laten lijken, met name in afgelegen gebieden.

2.3 Preliminare experimenten

Aangezien de Android Developer-website niet alle vragen beantwoordt en er nog veel onduidelijkheden zijn, is een deel van de state-of-the-art gewijd aan onderzoek naar hoe deze variabelen het uiteindelijke resultaat beïnvloeden.

2.3.1 Verschillende nauwkeurighedsniveaus

De Fused Location Provider biedt vier prioriteitsniveaus voor het bepalen van de locatie, zoals beschreven in de Android-documentatie:

- **Balanced power accuracy:** Gebruik deze instelling om locatieprecisie op te vragen binnen een stadsblok, wat een nauwkeurigheid van ongeveer 100 meter is. Dit wordt beschouwd als een grove nauwkeurigheid en verbruikt waarschijnlijk minder stroom. Met deze instelling maken de locatieservices waarschijnlijk gebruik van Wi-Fi en positionering van zendmasten. Houd er echter rekening mee dat de keuze van de locatieprovider afhankelijk is van veel andere factoren, zoals welke bronnen beschikbaar zijn.
- **High accuracy:** Gebruik deze instelling om de meest nauwkeurige locatie op te vragen. Met deze instelling is de kans groter dat de locatieservices GPS gebruiken om de locatie te bepalen.
- **Low power:** Gebruik deze instelling om precisie op stadsniveau aan te vragen, wat een nauwkeurigheid van ongeveer 10 kilometer is. Dit wordt beschouwd als een grove nauwkeurigheid en verbruikt waarschijnlijk minder stroom.
- **Passive:** Gebruik deze instelling als u een verwaarloosbare invloed op het stroomverbruik nodig heeft, maar locatie-updates wilt ontvangen wanneer deze beschikbaar zijn. Met deze instelling activeert uw app geen locatie-updates, maar ontvangt u locaties die door andere apps worden geactiveerd.

Naast de beschreven verschillen in nauwkeurigheid en energieverbruik, zijn er ook andere factoren die deze instellingen beïnvloeden. Uit experimenten bleek bijvoorbeeld dat de instelling *balanced power accuracy* wordt beïnvloed door de *doze*-modus of energiebesparingsmodus van Android. Tijdens tests werd vastgesteld dat een actief gebruikte telefoon een consistente locatie-trace genereerde, terwijl drie andere telefoons aanzienlijke hiaten vertoonden. Deze hiaten werden veroorzaakt doordat de Fused Location Provider de callback niet activeerde vanwege de energiebesparende staten.

Tests met de prioriteiten *low power* en *passive* werden niet uitgevoerd. De *low power*-instelling was niet relevant voor onderzoek naar gebruikerslocatie-traces, terwijl de *passive*-instelling te veel externe variabelen bevatte, waardoor geen waardevolle resultaten konden worden verkregen.

2.3.2 Coarse location Android

Om de coarse location-functionaliteit te onderzoeken, werd de applicatie tweemaal geïnstalleerd op dezelfde telefoon: één keer met coarse location-permissies en één keer met fine location-permissies. Beide applicaties waren geconfigureerd met dezelfde prioriteitsinstellingen om een eerlijke vergelijking mogelijk te maken. Een van de onderzoeksvragen was of de instelling

balanced power accuracy, die geen gebruik maakt van satellieten, dezelfde resultaten zou opleveren als coarse location. Dit zou inzicht geven in de vraag of de obfuscatie uitsluitend voortkomt uit de inherente onnauwkeurigheid van de gebruikte technologieën, of dat Android actief een extra laag van obfuscatie toepast op de locatiegegevens. Als resultaat vinden we de volgende visualisatie 2.3. Hierbij is rood de trace met fine probleem en blauw coarse permissies. Waaruit we kunnen concluderen dat bovenop de inherente onnauwkeurigheid ook nog extra obfuscatie gebeurt. Een interessante vraag die voortkwam uit de experimenten, was hoe de obfuscatie van Android precies werkt. Om dit te onderzoeken, werden de coarse location-gegevens van eerdere tests naast elkaar gelegd. Zoals te zien is in Figuur 2.4, leveren verschillende dagen en traces steeds dezelfde coördinaten op. Dit wijst erop dat Android een soort privacycirkels implementeert, vergelijkbaar met eerder besproken algoritmen, om aanvallen te voorkomen waarbij gemiddelden over lange periodes worden gebruikt om plaatsen van interesse te identificeren.

Een vervolgonderzoek richtte zich op de consistentie van dit algoritme. De vraag was of de geobfusceerde locaties consistent worden berekend op basis van nabijgelegen telefoonmasten, zodat dezelfde locaties op verschillende apparaten identiek zouden zijn. Uit de resultaten, weergegeven in Figuur 2.5, blijkt echter dat dit niet het geval is. Dit suggereert dat Android's obfuscatie-algoritme niet uitsluitend gebaseerd is op vaste berekeningen van telefoonmast-locaties, maar mogelijk andere factoren in overweging neemt. Uit de verzamelde data van eerdere experimenten bleek dat de gegenereerde locaties met coarse location-permissies op verschillende apparaten identieke resultaten opleverden. Dit riep de vraag op of deze locaties werden berekend op basis van telefoonmasten. Om dit te onderzoeken, werden alle telefoonmasten in Vlaanderen in kaart gebracht en vergeleken met de verzamelde locatiegegevens. Hoewel dit een interessante invalshoek biedt, valt verder onderzoek naar dit onderwerp buiten de scope van deze thesis.

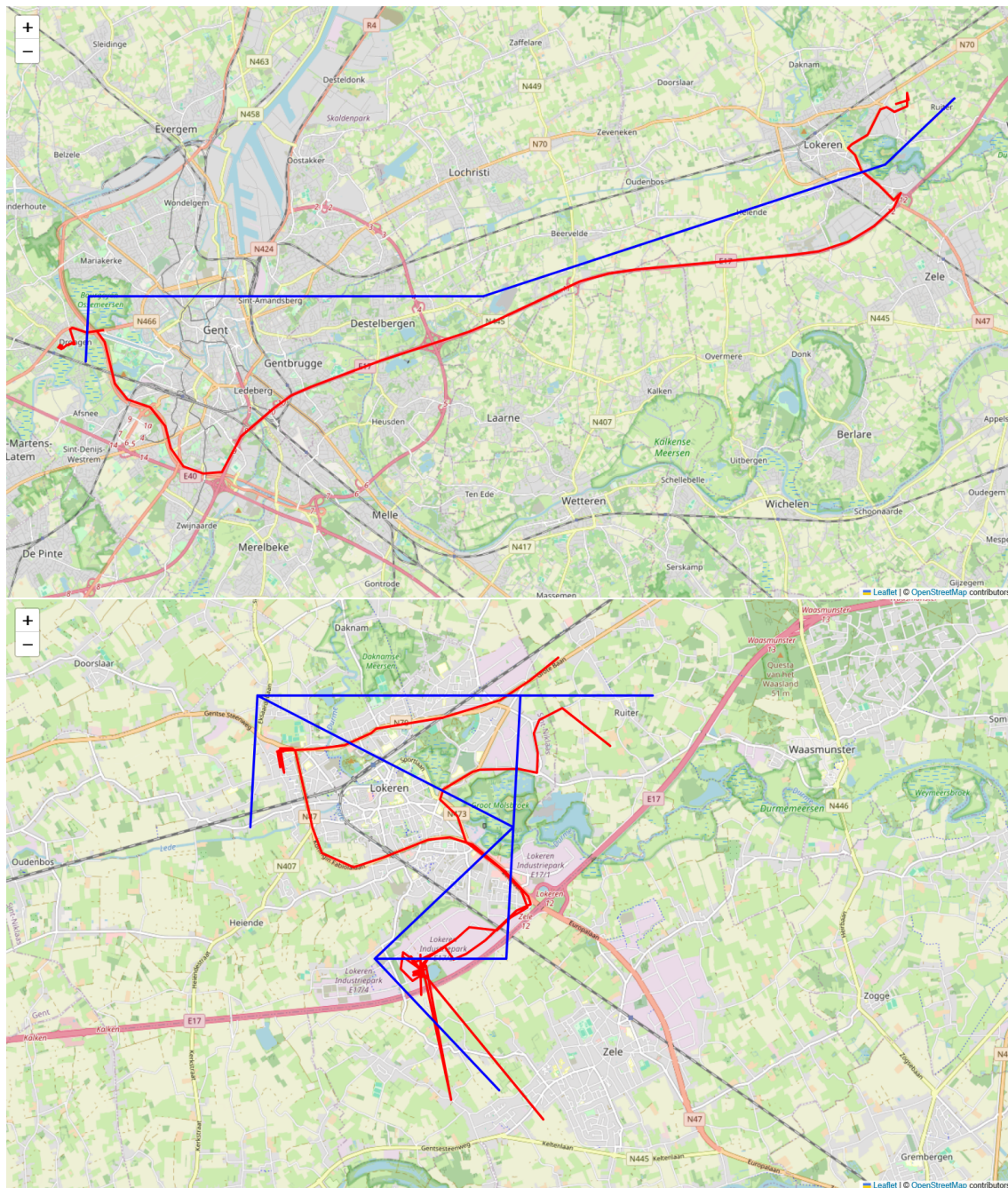


Figure 2.3: Vergelijking coarse permissies en fine permissies

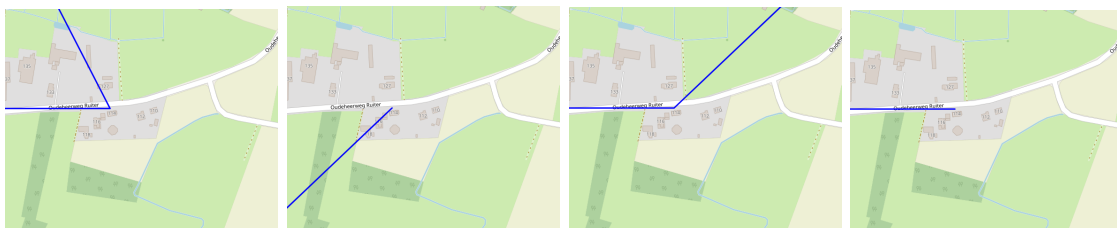


Figure 2.4: Vergelijking coarse permissie op één apparaat

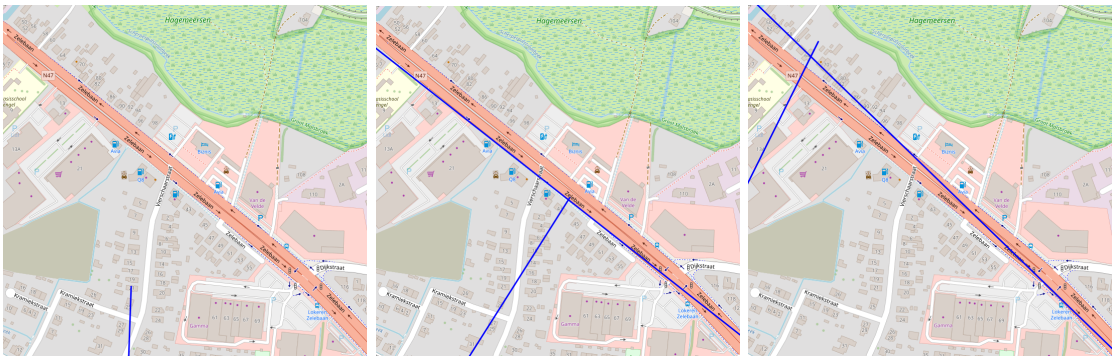


Figure 2.5: Vergelijking coarse permissie op meerdere apparaten

3

Design

Vooraleer de implementatie van de library kan starten, is een grondig ontwerp noodzakelijk. Dit ontwerp wordt in belangrijke mate bepaald door de beperkingen en mogelijkheden van het Android-platform. Daarnaast zijn de functionele vereisten en onderliggende veronderstellingen essentieel voor het uiteindelijke ontwerp.

3.1 Vereisten

De doelstellingen van dit onderzoek zijn als volgt geformuleerd: 1. Het ontwikkelen van een library met algemene bruikbaarheid: Het streven is om een library te creëren die in diverse applicaties kan worden toegepast. Deze library moet het proces van het aanvragen van permissies, initialisatie, gebruik en uitbreiding zo efficiënt mogelijk ondersteunen. Dit impliceert dat de library de volgende dingen moet zijn om in uiteenlopende soorten applicaties te kunnen werken.

- **Flexibel:** De library mag niet beperkt zijn tot de standaardinstellingen van de Android-implementatie en moet voldoende configuratiemogelijkheden bieden.
- **Modulair:** Gebruikers moeten in staat zijn om hun eigen obfuscatiealgoritmen te ontwikkelen en te integreren.
- **Eenvoudig te integreren:** De library moet functioneren met minimale setup en gebruiksvriendelijke integratiemogelijkheden bieden.
- **Beginnersvriendelijk:** Een van de aannames is dat de ontwikkelaar mogelijk weinig tot

geen kennis heeft van locatie services in Android. Daarom moeten de standaardinstellingen een solide basis bieden voor het veilig omgaan met locatiegegevens.

2. Op het vlak van privacy is de vereiste dat alle kwetsbare data het apparaat niet verlaat en dat de geobfusceerde locatie nog steeds bruikbaar is maar het niet toelaat een individu te traceren. Voor het aspect van obfuscatie vallen we terug op het werk van Toon Dehaene. Hierbij werd er gebruik gemaakt van ruis op de coördinaten en privacy cirkels. De cirkels zorgen er voor dat er geen gemiddelde kan genomen worden van de locaties om zo toch point of interest te kunnen verzamelen.

Deze technieken worden gecombineerd om een balans te vinden tussen privacy en bruikbaarheid, waarbij de gebruiker beschermd blijft tegen potentiële privacy-inbreuken. Door deze doelen te realiseren, wordt niet alleen de bruikbaarheid van de ontwikkelde oplossingen vergroot, maar wordt ook de drempel voor toekomstig onderzoek verlaagd.

3.2 Veronderstellingen

Om de werking van de library te kunnen garanderen, worden de volgende veronderstellingen gemaakt:

- **Goede intenties van de ontwikkelaar:** Er wordt aangenomen dat de ontwikkelaar te goeder trouw handelt. De library kan immers aangepast of volledig omzeild worden, waardoor het mogelijk is om alsnog de echte locatie door te sturen.
- **Integriteit van het Android-systeem:** Het wordt verondersteld dat de Android-installatie waarop de applicatie draait niet gecompromitteerd is. Dit betekent dat er geen derde partij of de eigenaar roottoegang heeft, aangezien dit zou toelaten om privacygevoelige gegevens rechtstreeks uit te lezen.
- **Correct gebruik van onderzoeks- en debugmodus:** De applicatie zal een onderzoeks- en debugmodus bevatten. Het is essentieel dat deze modus niet in productieomgevingen wordt gebruikt, omdat dit zou kunnen leiden tot het versturen van de echte locatie samen met de geobfusceerde locatie.
- **Gebruikersanonimiteit:** De permanente opslag van gegevens wordt gekoppeld aan de installatie van de applicatie in plaats van aan een specifieke gebruiker. Dit voorkomt dat locaties aan een gebruiker worden gelinkt en maakt het onmogelijk om door middel van herhaalde installaties en verwijderingen een gemiddelde van locaties te berekenen.

3.3 Ontwerp

Het ontwerp van de library is gebaseerd op het principe van een facade, waarbij de library fungeert als een vereenvoudigde interface voor de standaard Android-functionaliteiten. Dit

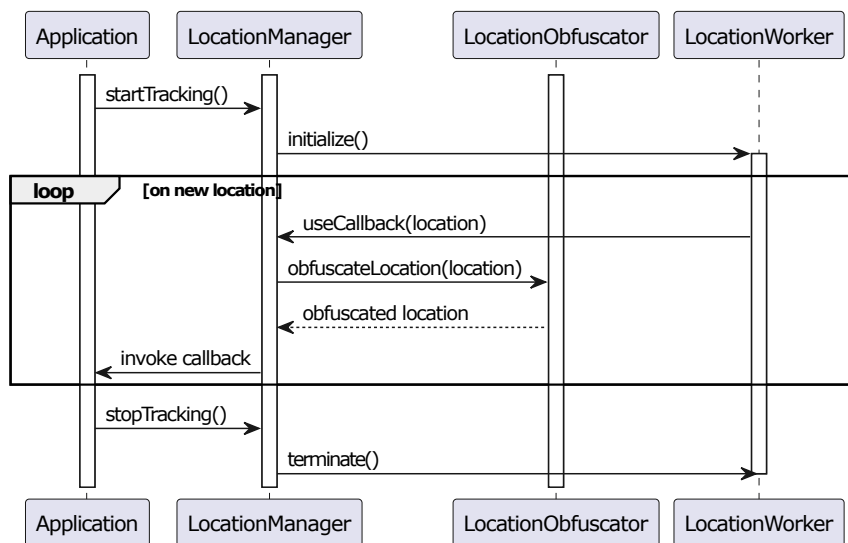


Figure 3.1: Flow diagram van continue locatieverzameling

betekent dat alle variabelen en functionaliteiten aanwezig moeten zijn alsof de ontwikkelaar rechtstreeks met de standaard Android-implementatie werkt, maar met de toegevoegde voordelen van de library. Met bovenstaande ontwerpprincipes in gedachten illustreert Figuur 3.1 hoe één enkele functie het verzamelen van locatiegegevens kan starten. Het is belangrijk op te merken dat deze implementatie de functie `RequestPermission` niet omvat, aangezien deze expliciet door de ontwikkelaar moet worden geïmplementeerd.

Verder is de `LocationManager` een Singleton gemaakt. Singletons zorgen ervoor dat er slechts één instantie van een klasse bestaat binnen de applicatie. Dit is er voor gekozen zodat als de developer twee-maal de `LocationManager` probeert te initialiseren de applicatie niet eventueel crashed doordat beide instanties locaties opvragen en het limiet overschrijden.

Daarnaast moeten alle standaardinstellingen en functionaliteiten van de library een solide basis bieden, zodat ontwikkelaars zonder uitgebreide kennis van Android-locatieservices de library direct kunnen gebruiken. Dit omvat het bieden van een goede standaardimplementatie die flexibel genoeg is om aanpasbaar te zijn voor meer geavanceerde toepassingen. Ook moeten de setters rekening houden met de grenzen van het Android systeem. Met als voorbeeld als de applicatie enkel coarse permissies heeft mag de app niet frequenter dan één keer om de 2 minuten een locatie vragen of de volledige `FusedLocationProvider` zal niet meer werken.

4

Implementatie

De eerste stap in deze thesis was het onderzoeken van hoe Android omgaat met locatiegegevens en het implementeren van deze functionaliteit in een testapplicatie. Zodra deze applicatie functioneel was, werden de eerste tests uitgevoerd en werd de nuttige code afgesplitst om de basis te vormen van de Android-library.

4.1 Overzicht library

De structuur van de library is weergegeven in Figuur 4.1. Het merendeel van de functionaliteit is ondergebracht in de klasse `LocationManager`, die fungeert als het centrale beheerpunt voor locatieverzameling. De klasse `LocationWorker` vormt de implementatie die vereist is om de `WorkManager` te benutten voor achtergrondtaken. Daarnaast biedt de interface `LocationObfuscator` een abstractielaag waarmee ontwikkelaars eigen obfuscatiealgoritmen kunnen implementeren.

4.1.1 Locatie tracking

Het centrale onderdeel van de library, namelijk de locatieverzameling, wordt beheerd door de `LocationManager`, die fungeert als het centrale aanspreekpunt. Deze klasse beheert alle benodigde variabelen voor locatieverzameling en biedt setters met ingebouwde validaties om de consistentie te waarborgen. Variabelen die specifiek zijn voor obfuscatie worden echter beheerd door de obfuscator zelf, aangezien deze buiten de verantwoordelijkheid van de `LocationManager` vallen.

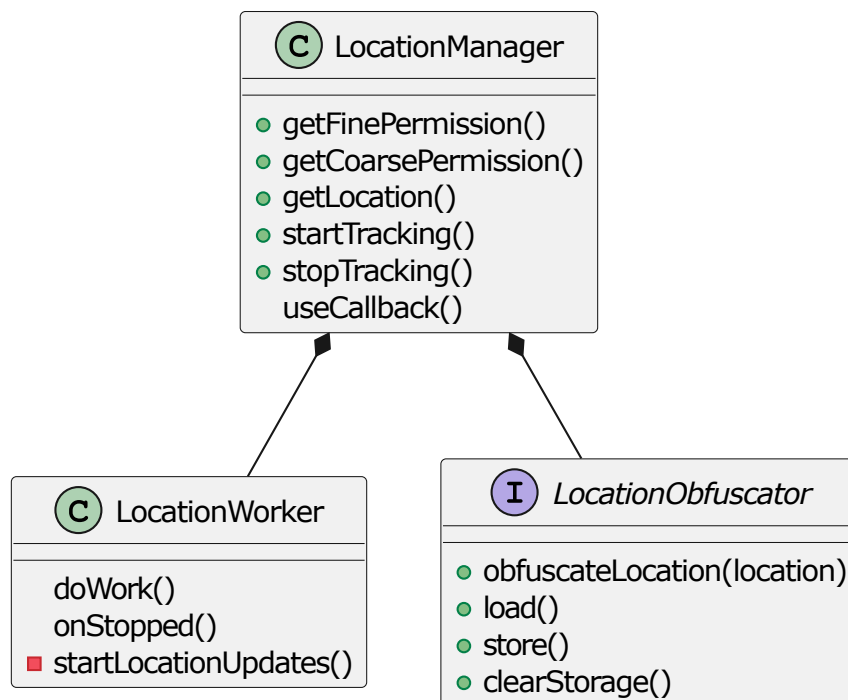


Figure 4.1: Klasse diagram van de library

4.1.1.1 background en foreground services

Voordat locaties verzameld kunnen worden, moeten enkele beperkingen in acht worden genomen. Allereerst mogen locaties niet worden opgevraagd in dezelfde thread als de activity waarmee de gebruiker interacteert. Zelfs als dit in een aparte thread vanuit de hoofdactivity wordt opgestart, zal Android deze beëindigen vanwege de strikte beperkingen op services wanneer de activity wordt gestopt of vernietigd.

Om taken uit te voeren wanneer de hoofdactivity niet actief is, kan gebruik worden gemaakt van foreground services of background tasks. Voor deze library is gekozen voor de WorkManager, vanwege de flexibiliteit en betrouwbaarheid die deze biedt. De WorkManager kan zowel functioneren als een foreground service als een periodieke background task, wat het beheer van locatieverzoeken aanzienlijk vereenvoudigt.

Foreground services Een foreground service wordt gebruikt om langdurige taken uit te voeren die een hogere prioriteit vereisen. Om een service als foreground service te configureren, moet in de hoofdactivity een notificatiekanaal worden aangemaakt. Dit notificatiekanaal zorgt ervoor dat de gebruiker visueel geïnformeerd wordt over de achtergrondactiviteit. Nadat het notificatiekanaal is aangemaakt, kan de Worker, een klasse die de vereiste interface van de WorkManager implementeert, een notificatie genereren en weergeven.

Zodra de notificatie is geactiveerd, kan de Worker ononderbroken blijven werken totdat deze zichzelf beëindigt of totdat de functie `cancelUniqueWork` van de WorkManager wordt

aangeroepen vanuit de hoofdactivity. Deze aanpak zorgt ervoor dat de taken betrouwbaar en efficiënt worden uitgevoerd, zelfs wanneer de applicatie zich niet op de voorgrond bevindt.

Background tasks Wanneer een visuele indicatie van achtergrondactiviteit niet vereist is, kan gebruik worden gemaakt van periodieke taken. Historisch gezien werd hiervoor de `AlarmManager` gebruikt, die functioneerde als een soort cron job in Linux. Echter, vanwege strengere beperkingen op maximale duur en frequentie, is deze aanpak minder geschikt geworden.

De `WorkManager` biedt een modern alternatief voor het uitvoeren van periodieke taken. Deze heeft echter ook enkele beperkingen: taken kunnen maximaal elke 15 minuten worden uitgevoerd en mogen niet langer dan 10 minuten actief zijn. Ondanks deze restricties biedt de `WorkManager` een robuuste en flexibele oplossing voor het beheren van achtergrondprocessen.

Integratie in de library Doordat de `WorkManager` beide vereisten kan afhandelen, is er slechts één klasse nodig die zowel foreground services als periodieke taken implementeert. Deze klasse, die de `Worker`-interface implementeert, bevat tevens de `LocationCallback` die vereist is voor de `FusedLocationProvider`. Deze callback speelt alle resultaten direct door naar de `LocationManager`, die deze resultaten vervolgens verwerkt volgens de ingestelde configuratie. Deze aanpak minimaliseert de complexiteit en verhoogt de herbruikbaarheid van de code.

Een bijkomende reden om de `LocationManager` als een Singleton te implementeren, is de beperking van de `LocationWorker` om complexe structuren of referenties door te geven. Door gebruik te maken van een Singleton-patroon kan de `LocationWorker` de `LocationManager` initialiseren en een referentie behouden naar diens variabelen. Zo bevat de `LocationManager` bijvoorbeeld een boolean genaamd `running`, waarmee kan worden aangegeven of de locatieverzameling actief is. Indien de locatieverzameling wordt stopgezet, kan de `LocationWorker` deze status uitlezen en zichzelf correct beëindigen.

4.1.1.2 Permissies

voordeel van Android is sandboxing, waardoor alles waar de applicatie probeert toegang tot te krijgen gemonitord kan worden. Hierdoor moet de applicatie toestemming vragen voor vele dingen anders als de applicatie probeert toegang te krijgen zal de applicatie crashen. permissies zijn verdeeld in 2 categorieën. Degene die in de manifest van de Android applicatie kunnen vermeld worden en er geen nood is om rechtstreeks toestemming aan de gebruiker te vragen.

De andere moet ook in de manifest vermeld worden maar moet ook vanuit de hoofdactiviteit opgevraagd worden met de functie `RequestPermission` waarbij Android dan visueel de permissie vragen afhandelt. Het is echter belangrijk op te merken dat deze functie pas kan worden aangeroepen nadat de activity is gecreëerd. Voor deze applicatie zijn de permissies

als volgt:

- Enkel manifest: `ACCESS_BACKGROUND_LOCATION`, `FOREGROUND_SERVICE` en `FOREGROUND_SERVICE_LOCATION`
- Met `RequestPermission`: `ACCESS_COARSE_LOCATION` en `ACCESS_FINE_LOCATION`

4.1.2 Obfuscatie

Wanneer de `Worker` de callback van de `LocationManager` aanroept met het locatie-resultaat van de `FusedLocationProvider`, worden de coördinaten en de timestamp uit dit resultaat geëxtraheerd.

Vervolgens wordt de obfuscatiefunctie van de ingestelde obfuscatieklasse aangeroepen. Indien er geen obfuscatieklasse is ingesteld, wordt er geen obfuscatie toegepast en fungeert de library als een eenvoudige facade. De resulterende, al dan niet geobfusceerde, locatie wordt daarna doorgegeven aan de callbackfunctie die door de ontwikkelaar is ingesteld. Het is echter ook mogelijk dat de obfuscatiefunctie een `null`-waarde retourneert. In dat geval wordt de callbackfunctie van de ontwikkelaar niet aangeroepen.

Daarnaast biedt de library ook de mogelijkheid om een debugcallback in te stellen. Indien een obfuscatieklasse aanwezig is, zal deze debugcallback worden aangeroepen met zowel de originele locatie als de geobfusceerde locatie. Dit biedt ontwikkelaars extra mogelijkheden voor analyse en onderzoek.

4.1.2.1 Standaard obfuscatiealgoritme

De library vereist een standaardimplementatie voor een obfuscatieklasse. Hiervoor wordt verwezen naar het werk van Toon Dehaene, waarin het concept van privacyzones centraal staat. Het toevoegen van enkel ruis aan locatiegegevens kan immers leiden tot het risico dat een aanvaller door middel van gemiddelden alsnog gevoelige informatie kan achterhalen. Het algoritme, zoals weergegeven in Figuur 4.2, illustreert de werking van deze aanpak.

Het proces start met het filteren van de lijst van bestaande privacycirkels om te bepalen of de nieuwe locatie binnen een van deze cirkels valt. Hierbij wordt tevens de privacycirkel bijgehouden waarvan het middelpunt het dichtst bij de huidige locatie ligt. Indien geen enkele cirkel aan de criteria voldoet, wordt een nieuwe privacycirkel aangemaakt. Het middelpunt van deze nieuwe cirkel wordt willekeurig gekozen binnen een vooraf ingestelde straal rondom de nieuwe locatie en de straal van deze cirkel is ook deze ingestelde straal.

Daarnaast bevat het algoritme een mechanisme om te voorkomen dat locaties te frequent worden verzonden. Indien een locatie kort na een eerdere verzending wordt gegenereerd, retourneert de obfusculator een `null`-waarde. Dit voorkomt overbodige updates, ook dit is in te stellen door de developer.

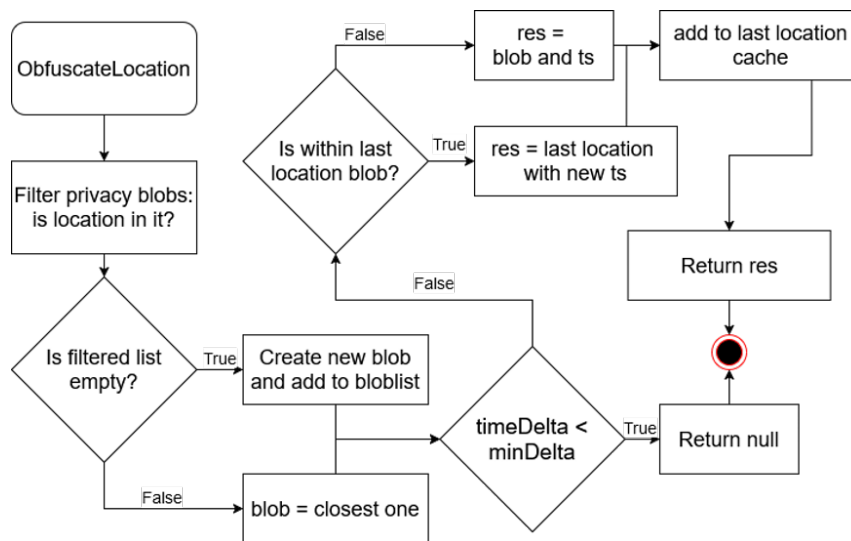


Figure 4.2: Flow diagram van het obfuscatie algoritme

Om jitter te vermijden—waarbij de obfuscator blijft schakelen tussen twee overlappende privacycirkels en daardoor een nauwkeurigere locatie prijsgeeft—wordt een extra controle uitgevoerd. Indien de gebruiker zich nog steeds binnen de laatst gebruikte privacycirkel bevindt, wordt deze cirkel opnieuw gebruikt voor de obfuscatie. Dit mechanisme draagt bij aan een consistente en veilige verwerking van locatiegegevens.

4.1.3 Opslag

Omdat veel obfuscatiealgoritmen permanente opslag nodig heeft moet iedere obfuscatieklasse ook een load en store functie implementeren. Dit heeft als argument de filesDir van de hoofdactiviteit. In de standaardimplementatie wordt dit gebruikt om voorgaande privacy cirkels op te slaan in een file afgesloten voor andere applicaties.

4.1.4 Netwerk

De library heeft geen functionaliteit voor de locatie door te sturen naar een API endpoint. Dit moet geïmplementeerd worden in de callback function die de developer meegeeft aan de LocationManager. In de testapplicatie word de library, aanbevolen door Android zelf, okhttp gebruikt om de locatie door te sturen.

4.2 Dataverwerking

4.2.1 Verzameling

Voor de verzameling van data werd gebruik gemaakt van een Azure-webserver, specifiek opgezet voor studenten, met twee endpoints: `location` en `blobs`.

De `location`-endpoint ontving zowel de geobfusceerde als de originele locatiegegevens, afkomstig van de `debugcallback`, samen met relevante configuratieparameters. Deze gegevens werden opgeslagen in een CSV-bestand voor latere analyse.

De `blobs`-endpoint werd gebruikt om de privacycirkels door te sturen.

Om de afhankelijkheid van een continue internetverbinding te minimaliseren, werd een mechanisme geïmplementeerd waarbij de data lokaal werd opgeslagen wanneer er geen netwerkverbinding beschikbaar was. Zodra de verbinding werd hersteld, werden de opgeslagen gegevens alsnog doorgestuurd naar de server.

4.2.2 verwerking

Voor de verwerking van de resultaten werden jupyter notebooks gehanteerd samen met de library *Folium* om de resultaten visueel te kunnen representeren. De eerste visualisatie was het verschil tussen coarse en fine Permissies vergelijken voor een experiment, dit was beide gedaan op balanced power accuracy gebeurd zodat beide geen gps satellieten gebruikt. Het resultaat is te zien in het hoofdstuk experimenten in state of the art 2.3. Een tweede test betrof het gebruik van twee telefoons met coarse permissies en twee met fine permissies. Voor elk paar werd één telefoon ingesteld op high accuracy en de andere op balanced power accuracy. Deze test werd echter niet opgenomen in het experimentenonderdeel van het hoofdstuk *State of the Art*, omdat er te veel externe factoren waren die de resultaten beïnvloedden, zoals de beperkingen opgelegd door de energiebesparingsfunctionaliteiten van Android. De overige tests richtten zich op het evalueren van de standaardimplementatie van het obfuscatiealgoritme. Hierbij werden verschillende variabelen gevarieerd om de werking van het algoritme te analyseren en mogelijke verbeterpunten te identificeren. De inzichten die uit deze tests voortkwamen, werden gebruikt om de library verder te debuggen en te optimaliseren. Een uitgebreide bespreking van deze resultaten is te vinden in het hoofdstuk *Resultaten*.

4.3 Conclusie

In dit hoofdstuk werd de implementatie van de Android-library uitgebreid besproken. De library is ontworpen om locatiegegevens op een privacyvriendelijke manier te verzamelen en te verwerken. Het centrale beheer van locatieverzameling wordt uitgevoerd door de `LocationManager`, terwijl de `WorkManager` zorgt voor een flexibele en betrouwbare uitvoering van zowel foreground services als periodieke background tasks.

De library integreert een standaard obfuscatiealgoritme gebaseerd op privacycirkels, waarmee gevoelige informatie effectief wordt beschermd. Ontwikkelaars hebben de mogelijkheid om aangepaste obfuscatieklassen te implementeren en gebruik te maken van debugcallbacks voor analyse. Permanente opslag wordt ondersteund om privacycirkels en andere gegevens veilig te bewaren.

Daarnaast werd een testapplicatie ontwikkeld om data te verzamelen via een Azure-webserver. Hierbij werden mechanismen geïmplementeerd om gegevens lokaal op te slaan bij netwerkuitval, wat de betrouwbaarheid van de oplossing verhoogt. Deze aanpak minimaliseert de afhankelijkheid van een continue internetverbinding.

5

Resultaten

Naast de resultaten van de testen in verband met de werking van Android besproken in de state of the art zijn dit de resultaten van de library en zijn standaardimplementatie van een obfuscatiealgoritme.

5.1 Visuele representatie

In 5.1 zien we visueel de werking van de standaardimplementatie van het obfuscatiealgoritme. De gekozen variabelen voor deze visualisatie is een privacycirkel met een straal van 400 meter en een tijds interval van 5 minuten.

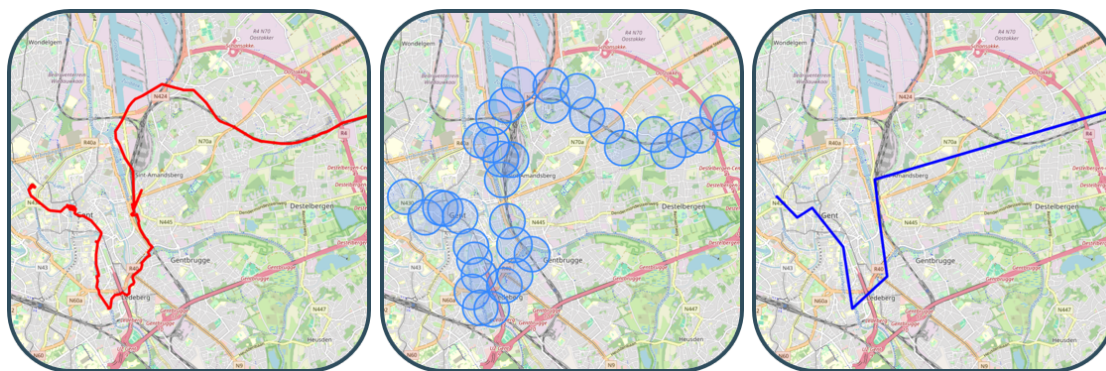


Figure 5.1: Obfuscatiealgoritme gevisualiseerd

6

Evaluatie en discussie

6.1 Evaluatie van het systeem

6.1.1 beperkingen

In de huidige versie van de library is slechts beperkt onderzoek verricht naar de langdurige stabiliteit van locatieverzameling in de achtergrond en naar mogelijke onbekende beperkingen die verder reiken dan de maximale threaddtijd van 10 minuten per 15 minuten. Om dit grondig te testen, zouden meerdere langdurige runtimes moeten worden voorzien.

Voor de testapplicatie is een fallbackmechanisme geïmplementeerd voor mislukte HTTP-verzoeken, waarbij deze tijdelijk worden opgeslagen in de shared preferences van Android. Echter, shared preferences zijn niet geschikt voor het verwerken van grote hoeveelheden data, wat leidt tot vertragingen bij het lezen en schrijven, vooral wanneer er gedurende langere tijd geen netwerkverbinding beschikbaar is. Bovendien kan de plotselinge piek in internetverkeer bij het herstellen van een netwerkverbinding resulteren in crashes op Android. Verdere uitbreiding van de testapplicatie vereist daarom throttlingmechanismen en een robuustere permanente opslagoplossing, met name voor scenario's waarin testapparaten langdurig geen internetverbinding hebben.

6.2 privacy

6.3 Gebruik van de library

6.3.1 initialisatie

Het gebruik van de library verloopt in enkele duidelijke stappen. Eerst wordt een instantie van de `LocationManager` aangemaakt, waarbij de `MainActivity` als parameter wordt meegegeven. Dit is noodzakelijk om een observer te kunnen koppelen aan de lifecycle van de `MainActivity`. Vervolgens wordt een `LocationObfuscator` ingesteld; dit kan zowel de standaardimplementatie zijn als een door de ontwikkelaar zelf geïmplementeerde klasse. Ten slotte wordt een callbackfunctie geregistreerd, die door de library wordt aangeroepen zodra er een nieuwe locatie beschikbaar is.

```
class MainActivity : ComponentActivity() {
    private val locationManager = LocationManager.getInstance(this)
    init {
        locationManager.locationObfuscator = LocationObfuscatorV1.getInstance()
        locationManager.callback =
            { location -> sendLocationToServer(location) }
    }
}
```

6.3.2 LocationObfuscator zelf implementeren

Om een eigen locatie-obfuscator te implementeren, kan men onderstaande aanpak volgen. De eenvoudigste implementatie bestaat erin dat de locatie verkregen via de `FusedLocationProvider` als argument wordt doorgegeven aan de `obfuscateLocation`-functie, die vervolgens een tuple van coördinaten en tijdstempel retourneert. De overige drie functies (`load`, `store` en `clearStorage`) zijn bedoeld voor het beheren van permanente opslag. Indien nodig kunnen variabelen via de constructor worden meegegeven. Het `filesDir`-object dat aan deze functies wordt doorgegeven, biedt toegang tot de opslagruimte van de applicatie.

```
import android.location.Location
import be.kuleuven.milanschollier.safegps.LatLonTs
import be.kuleuven.milanschollier.safegps.LocationObfuscator
import java.io.File
class ImplementationExample private constructor() : LocationObfuscator {
    override fun obfuscateLocation(location: Location): LatLonTs {
        return LatLonTs(location.latitude, location.longitude, location.time)
    }
    override fun load(filesDir: File) {return}
```

```
    override fun store(filesDir: File) {return}  
    override fun clearStorage(filesDir: File) {return}  
}
```

6.3.3 Live data

De developer kan gebruik maken van live data in de library. Zo is `finePermission`, `coarsePermission` en `running live data` dat de applicatie intuïtief kan laten reageren op de werking van de library.

```
val finePermission = locationManager.finePermission.observeAsState()  
Button(  
    onClick = {  
        locationManager.getFinePermission(true)  
    },  
    modifier = Modifier  
        .weight(1f)  
        .fillMaxSize(),  
    enabled = finePermission.value == false  
) {  
    Text(text = if (finePermission.value == true) "Fine permission granted" else "Ask F  
}
```

6.4 Conclusie

7

Conclusie

Deze thesis demonstreert dat het mogelijk is om locatiegegevens on-device te obfusceren zonder de bruikbaarheid voor analyse significant aan te tasten. Door het ontwikkelen van een Android-bibliotheek die locaties en tijdstempels vervormt via ruis en privacycirkels, wordt voorkomen dat individuele gebruikers geïdentificeerd kunnen worden via hun bewegingspatroon. De kern van de oplossing ligt in het behoud van data-utility: geaggregeerde inzichten, zoals mobiliteitstrends of drukteanalyses, blijven mogelijk, terwijl gevoelige individuele informatie wordt beschermd.

Een belangrijk succes is de volledige onafhankelijkheid van externe servers. Alle verwerking vindt plaats op het apparaat zelf, waardoor de originele, nauwkeurige locatie nooit het toestel verlaat. Dit elimineert het risico op datalekken bij overdracht of opslag. Technisch werd dit gerealiseerd door een modulaire ontwerp, waarbij de bibliotheek naadloos integreert met Android's Fused Location Provider en WorkManager voor betrouwbare achtergrondverwerking. Daarnaast biedt de implementatie van een standaard obfuscatiealgoritme op basis van privacycirkels een evenwicht tussen anonymisering en nauwkeurigheid, geïnspireerd op eerder onderzoek.

De bibliotheek verlaagt niet alleen de implementatiedrempel voor ontwikkelaars door complexe privacytechnieken te abstraheren, maar draagt ook bij aan een paradigmaverschuiving in databeheer. In plaats van privacy als een afterthought te behandelen, wordt bescherming een inherent onderdeel van het ontwerp. Dit sluit aan bij groeiende regelgevende eisen en maatschappelijke vraag naar transparante datapraktijken.

Hoewel verdere optimalisatie mogelijk blijft, toont deze aanpak aan dat technische oplossingen een sleutelrol kunnen spelen in het verzoenen van data-innovatie en individuele rechten.

Door privacy-by-design toegankelijk te maken, zet deze thesis een stap richting een toekomst waarin technologie niet ten koste gaat van gebruikerscontrole, maar deze net versterkt.



Code van het obfuscatie algoritme

```
override fun obfuscateLocation(location: Location): LatLonTs? {
    val lat = location.latitude
    val lon = location.longitude
    val ts: Long = location.time

    val matchedReports = historyBlobs.filter { report ->
        Geodesic.WGS84.Inverse(
            report.first,
            report.second,
            lat,
            lon,
            GeodesicMask.DISTANCE
        ).s12 < report.third
    }

    val blob: PrivacyBlob
    if (matchedReports.isEmpty()) {
        val sampledRadius = radiusSampler()
        val sampledAngle = Math.toDegrees(angleSampler())
        val res = Geodesic.WGS84.Direct(lat, lon, sampledAngle, sampledRadius)
        blob = PrivacyBlob(res.lat2, res.lon2, _blobRadius)
        println("Generated new privacy blob: $blob")
        historyBlobs.add(blob)
    }
}
```

```

    } else {
        blob = matchedReports.minBy { report ->
            Geodesic.WGS84.Inverse(
                report.first,
                report.second,
                lat,
                lon,
                GeodesicMask.DISTANCE
            ).s12
        }
    }
}

val lastLocation = perturbedLocationCache.lastOrNull() ?: LatLonTs(0.0, 0.0, 0)

if (ts - lastLocation.third < _deltaTime * 1000) {
    return null
}
//Voor jitter tegen te gaan als in de buurt van vorige locatie die
//privacy blob doorsturen ongeacht andere dichters is
val res: LatLonTs = if (Geodesic.WGS84.Inverse(
    lastLocation.first,
    lastLocation.second,
    lat,
    lon,
    GeodesicMask.DISTANCE
).s12 < _blobRadius
) {
    LatLonTs(lastLocation.first, lastLocation.second, ts)
} else {
    LatLonTs(blob.first, blob.second, ts)
}

perturbedLocationCache.add(res)
if (perturbedLocationCache.size > 5) perturbedLocationCache.removeAt(0)
return res
}

```



Beschrijving van deze masterproef in de vorm van een wetenschappelijk artikel

The thesis should also contain a short scientific article. If you write your thesis in Dutch, you must write the article in English, and vice versa. We advise you to employ the IEEE Manuscript Templates for Conference Proceedings (<https://www.overleaf.com/latex/templates/ieee-conference-template/grfzhnncsfqn>). Compile the article in another project and include the generated pdf file as shown below:

Conference Paper Title*

*Note: Sub-titles are not captured in Xplore and should not be used

1st Given Name Surname

dept. name of organization (of Aff.)

name of organization (of Aff.)

City, Country

email address or ORCID

2nd Given Name Surname

dept. name of organization (of Aff.)

name of organization (of Aff.)

City, Country

email address or ORCID

3rd Given Name Surname

dept. name of organization (of Aff.)

name of organization (of Aff.)

City, Country

email address or ORCID

4th Given Name Surname

dept. name of organization (of Aff.)

name of organization (of Aff.)

City, Country

email address or ORCID

5th Given Name Surname

dept. name of organization (of Aff.)

name of organization (of Aff.)

City, Country

email address or ORCID

6th Given Name Surname

dept. name of organization (of Aff.)

name of organization (of Aff.)

City, Country

email address or ORCID

Abstract—This document is a model and instructions for $\LaTeX$. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.

Index Terms—component, formatting, style, styling, insert

I. INTRODUCTION

This document is a model and instructions for $\LaTeX$. Please observe the conference page limits.

II. EASE OF USE

A. Maintaining the Integrity of the Specifications

The IEEEtran class file is used to format your paper and style the text. All margins, column widths, line spaces, and text fonts are prescribed; please do not alter them. You may note peculiarities. For example, the head margin measures proportionately more than is customary. This measurement and others are deliberate, using specifications that anticipate your paper as one part of the entire proceedings, and not as an independent document. Please do not revise any of the current designations.

III. PREPARE YOUR PAPER BEFORE STYLING

Before you begin to format your paper, first write and save the content as a separate text file. Complete all content and organizational editing before formatting. Please note sections III-A–III-E below for more information on proofreading, spelling and grammar.

Keep your text and graphic files separate until after the text has been formatted and styled. Do not number text heads— $\LaTeX$ will do that for you.

Identify applicable funding agency here. If none, delete this.

A. Abbreviations and Acronyms

Define abbreviations and acronyms the first time they are used in the text, even after they have been defined in the abstract. Abbreviations such as IEEE, SI, MKS, CGS, ac, dc, and rms do not have to be defined. Do not use abbreviations in the title or heads unless they are unavoidable.

B. Units

- Use either SI (MKS) or CGS as primary units. (SI units are encouraged.) English units may be used as secondary units (in parentheses). An exception would be the use of English units as identifiers in trade, such as “3.5-inch disk drive”.
- Avoid combining SI and CGS units, such as current in amperes and magnetic field in oersteds. This often leads to confusion because equations do not balance dimensionally. If you must use mixed units, clearly state the units for each quantity that you use in an equation.
- Do not mix complete spellings and abbreviations of units: “Wb/m²” or “webers per square meter”, not “webers/m²”. Spell out units when they appear in text: “. . . a few henries”, not “. . . a few H”.
- Use a zero before decimal points: “0.25”, not “.25”. Use “cm³”, not “cc”.)

C. Equations

Number equations consecutively. To make your equations more compact, you may use the solidus (/), the exp function, or appropriate exponents. Italicize Roman symbols for quantities and variables, but not Greek symbols. Use a long dash rather than a hyphen for a minus sign. Punctuate equations with commas or periods when they are part of a sentence, as in:

$$a + b = \gamma \quad (1)$$

Be sure that the symbols in your equation have been defined before or immediately following the equation. Use “(1)”, not

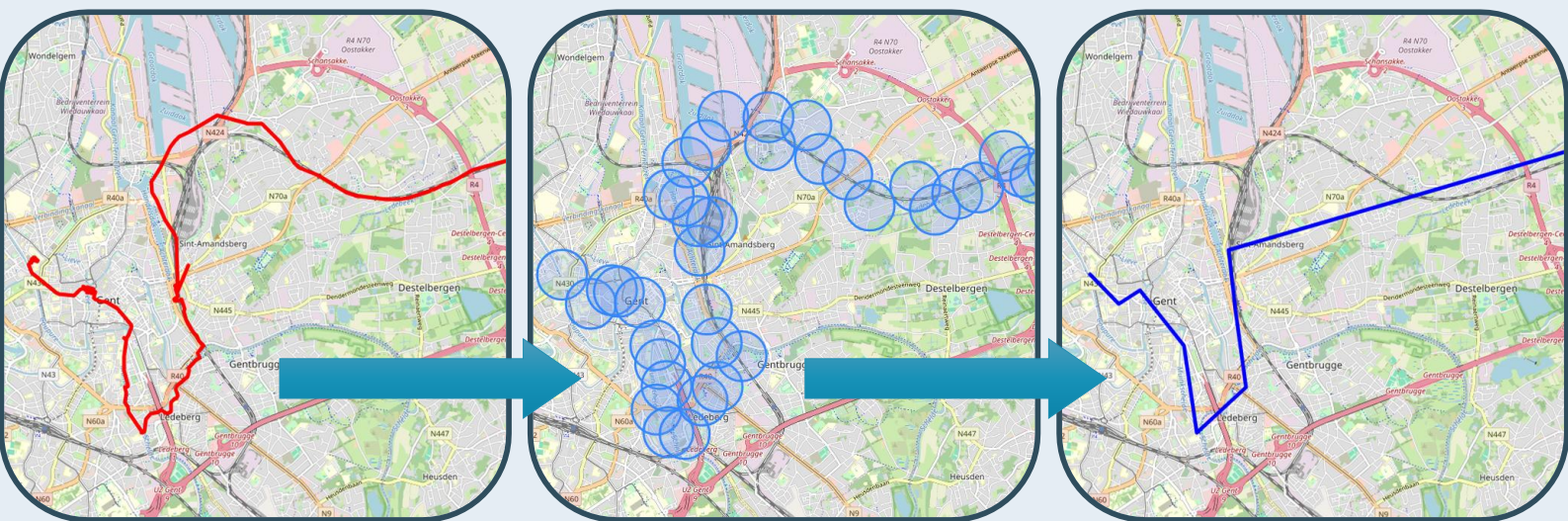
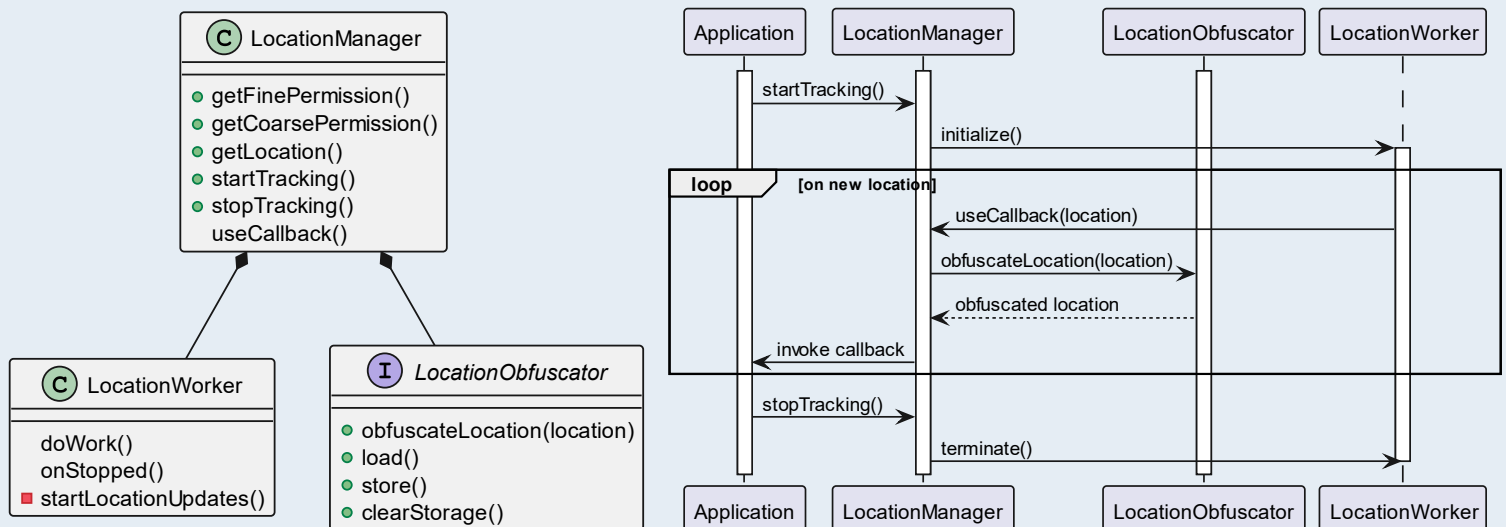
C

Poster

Ontwikkeling van een smartphone sdk voor het obfusceren van locatie data

Abstract

Locatiedata biedt waardevolle inzichten, maar brengt aanzienlijke privacy risico's met zich mee, zoals het afleiden van gevoelige locaties en sociale grafen. Deze thesis introduceert een innovatieve Android-bibliotheek die locatiedata obfusceert door ruis toe te voegen aan locaties en tijdstempels. Dit voorkomt dat individuele sporen aan gebruikers worden gekoppeld, terwijl de bruikbaarheid van de data behouden blijft. De bibliotheek werkt volledig op het apparaat, zodat nauwkeurige gegevens nooit het toestel verlaten. Experimenten tonen de effectiviteit aan en vergelijken de oplossing met Android 12's Approximate Location-functie, wat bijdraagt aan verbeterde mobiele privacy.



KU LEUVEN - GENT

Faculteit Industriële Ingenieurswetenschappen
Campus Rabot, Gebroeders De Smetstraat 1

9000 Gent, België

tel: + 32 9 265 86 10

iiw.gent@kuleuven.be

www.iw.kuleuven.be

